

The Engine Room, Part 1: मोजण्याची कला

August 30, 2026

THE ENGINE ROOM

Book 6, Part 1: मोजण्याची कला

हे पुस्तक कुणासाठी: जगातला प्रत्येक top engineer एक समान पाया शिकलेला असतो: algorithms, data structures, आणि यंत्र आतून कसं चालतं. Book 1 ने machine बाहेरून दाखवली; हे पुस्तक आत नेतं: पण आकडेमोड-गणिताने नाही, **तपासाच्या गोष्टीने**. का महत्वाचं? कारण दोन उत्तरं दोन्ही “बरोबर” असतात, पण एक 1000 पट हळू. **महागडं कुठलं हे ओळखणारी नजर** हीच top talent ची गाळणी: agent code लिहील, पण “हे हळू का” ओळखणं तुमचं.

या पुस्तकाचं एकमेव सूत्र

प्रत्येक हुशारी ही वेळ आणि जागा (memory) यांची अदलाबदल आहे.
फुकट वेग नसतो: कुठेतरी जागा किंवा मेहनत विकावी लागते.

Book 5 चा “सौदा” इथे यंत्राच्या तळाशी: वेगासाठी जागा विका, जागेसाठी वेग विका. जो हा सौदा बघतो तो चंद्रचं app वेगवान करतो; जो बघत नाही तो अंधारात चाचपडतो. प्रत्येक chapter शेवटी हे सूत्र परत भेटेल.

चंद्रचं app हळू आहे: या पुस्तकाची तपास-गोष्ट

पूर्ण पुस्तक एक **तपास** आहे. **चंद्र**, वय 24, ने एक साधं app बनवलं: गावातल्या लोकांची फोन-वही (नाव शोधा, नंबर मिळवा). सुरुवातीला भन्नाट चाललं. पण आता, 10 लाख नावं झाल्यावर, app **भयंकर हळू** झालंय: एक नाव शोधायला 8 सेकंद. चंद्र हैराण. आपण **गुप्तहेर** आहोत: प्रत्येक chapter मध्ये एका **संशयिताची** उलटतपासणी करू (कोण app हळू करतंय: चुकीचा algorithm? चुकीची रचना? disk? CPU? कुलूप?). प्रत्येक तपासातून एक CS- संकल्पना उलगडेल, आणि शेवटच्या chapter मध्ये **केस सुटेल**. चंद्रच्या जागी स्वतःला ठेवा: त्याचा प्रत्येक पेच तुमचा.

कपाटावर एक नजर

Book 1	The Machine	Book 2	The Thinking Machine
Book 3	The Land Game	Book 4	The Harness
Book 5	The Blueprint	Book 7	The Lens
Book 6	The Engine Room	<- हे: Computer च्या आत	

तीन भागांचा नकाशा: तुम्ही इथे आहात

PART 1	मोजण्याची कला	<- तुम्ही इथे
	Big-O, data structures:	महागड कुठलं ओळखणं
PART 2	यंत्राचा तळ	
	CPU, memory, OS:	यंत्र प्रत्यक्ष कसं चालतं
PART 3	पूल: एकाच वेळी अनेक कामं	
	concurrency, locks, distributed	चा पाया

चला पहिल्या संशयिताकडे: app 10 नावांना जलद, 10 लाखांना हळू: का? हे “किती वाढतं” चं गणित समजून घेऊ.

विभाग 1: मोजण्याची कला

तपासाची पहिली फेरी. चंदूचं app लहान असताना जलद, मोठं झाल्यावर हळू: म्हणजे संशयित “आकारासोबत कसं वागतं” यात आहे. हा विभाग ते मोजायला शिकवतो: Big-O (वाढीचा दर), आणि माहिती साठवायच्या वेगवेगळ्या रचना (data structures), प्रत्येकीचा वेगळा सौदा. **हे तुमच्या शिक्षणात कुठे येईल:** “हे code हळू का”, “कुठली रचना निवडू”, “agent ने बांधलेलं कार्यक्षम आहे का”: या प्रत्येक प्रश्नाचं उत्तर इथे. Top engineer आणि नवखा यांतला सर्वात मोठा फरक: महागडं कुठलं हे **आधीच** दिसणं.

Chapter 1.1 [SPINE]: संशायित पहिला: Big-O: वाढीचा दर

तपास सुरू. चंदूचं app 10 नावांना क्षणात, 10 लाखांना 8 सेकंद. पहिला प्रश्न गुप्तहेराचा: ”आकार वाढला की वेळ कसा वाढतो?” आणि हे मोजायचं एक साधं, शक्तिशाली साधन आहे: **Big-O**.

Big-O म्हणजे काय. “आकार दुप्पट केला, तर काम किती-पट वाढतं?” या एका प्रश्नाचं उत्तर. चहाच्या टपरीची उपमा: (1) एका गिन्हाईकाला चहा द्यायला ठराविक वेळ, गिन्हाईक 1 असो की 100: प्रत्येकाला तेवढाच (रांगेत पुढचा): हा **स्थिर** वेळ. (2) पण सगळ्या गिन्हाईकांची नावं मोठ्याने वाचायची, तर 100 गिन्हाईक = 100 पट वेळ: **रेषीय**. (3) आणि प्रत्येक गिन्हाईकाला **इतर प्रत्येकाशी** ओळख करून द्यायची? 100 गिन्हाईक = $100 \times 100 = 10,000$ ओळखी: **चौरस** (भयंकर). Big-O हे नेमकं हे मोजतं: काम आकारासोबत किती वेगात फुगतं.

चार महत्त्वाचे दर (चंदूला ओळखायलाच हवेत):

$O(1)$	स्थिर: आकार कितीही, वेळ तोच. (स्वप्न.) उदा: यादीतल्या 5व्या नावाला थेट बोट.
$O(\log n)$	लॉग: आकार दुप्पट, वेळ फक्त एक पाऊल वाढतो. (अप्रतिम.) उदा: अर्ध-अर्ध कापत शोध (1.7).
$O(n)$	रेषीय: आकार दुप्पट, वेळ दुप्पट. (ठीक.) उदा: पूर्ण यादी एकदा चाळणं.
$O(n^2)$	चौरस: आकार दुप्पट, वेळ चौपट. (धोका!) उदा: प्रत्येकाची प्रत्येकाशी तुलना.

चंदूचा संशायित सापडतो. चंदूचं app नाव शोधायला **पूर्ण यादी पहिल्यापासून चाळत** होतं ($O(n)$: Book 5, 1.3 चा full scan). 10 नावं = 10 पावलं (क्षणात); 10 **लाख** नावं = 10 लाख पावलं (8 सेकंद!). संशायित पकडला: **$O(n)$ शोध, मोठ्या यादीवर.** आणि आता Big-O ची जादू: चंदूला उपाय दिसतो: जर शोध $O(\log n)$ केला (अर्ध-अर्ध कापत: 1.7), तर 10 लाख नावं फक्त ~20 पावलांत (8 सेकंद -> milliseconds). **तोच data, वेगळा दर, 1000 पट वेग.** हेच “महागडं ओळखणं.”

सर्वात मोठा धडा: लहान आकारात सगळं सारखंच दिसतं. 10 नावांना $O(n)$ आणि $O(\log n)$ दोन्ही क्षणात: फरक दिसतच नाही. म्हणून चंदूने सुरुवातीला काळजी केली नाही (योग्यच: Book 5 चा premature-optimization टाळा). पण आकार वाढल्यावर दर उघड झाला. **Big-O चा फायदा:** **आकार वाढण्याआधीच “हे मोठं झालं की मरेल का” ओळखणं.** नवखा लहान-आकारात test करून “चालतंय” म्हणतो; जाणकार Big-O बघून “10 लाखांना काय होईल” आधीच सांगतो.

इथे लोक काय चुकीचं समजतात

”माझ्या test मध्ये जलद चाललं, म्हणजे code कार्यक्षम.” लहान data वर **सगळे** algorithms जलद (10 नावांना $O(n^2)$ सुद्धा क्षणात). “चाललं” म्हणजे “मोठ्यावरही चालेल” नव्हे. आणि उलट गैरसमज: “नेहमी सर्वात कमी Big-O हवा.” नाही: लहान/न-वाढणाऱ्या data ला साधा $O(n)$ पुरतो, आणि तो लिहायला-समजायला सोपा (Book 5 चा सौदा). Big-O ची काळजी **आकार मोठा होणार असेल तरच**. “किती वाढणार” आधी विचारा, मग दर निवडा.

नकाशावर

(केसची प्रगती दर chapter ला एक ओळ.)

1.1 Big-O = वाढीचा दर (आकार दुप्पट -> वेळ किती-पट); $O(1)/O(\log n)/O(n)/O(n^2)$; चंदूचा app = $O(n)$ शोध, म्हणून हळू

सूत्रावर: Big-O हा पहिला चष्मा: “महागडं कुठलं” हे आकार-वाढण्याआधीच दाखवतो. आणि उपाय नेहमी सौदा: $O(n)$ वरून $O(\log n)$ ला जायला बहुधा जास्त **जागा** (रचना, index) लागते: वेग विकत, जागा विकून. फुकट वेग नाही.

स्वतः बघा (5 मिनिटं)

फोनबुक (खरं, कागदी) घ्या. एक नाव शोधा: तुम्ही पहिल्या पानापासून चाळत नाही ($O(n)$); तुम्ही **अंदाजे मध्ये उघडता, अर्ध-अर्ध कापत** जाता ($O(\log n)$): नाव आधीच्या अक्षरात? मागे; पुढच्या? पुढे. म्हणून हजार पानी फोनबुकातही नाव सेकंदात. तुम्ही रोज $O(\log n)$ जगता; चंदूच्या app ला तेच शिकवायचंय.

विचार करा

चंदूला उपाय दिसला ($O(\log n)$ शोध), पण अडचण: “अर्ध-अर्ध कापत शोधायला यादी **क्रमाने लावलेली** हवी (फोनबुक अकारविल्हे असतं म्हणून). पण माझी नावं एका साध्या यादीत, कशीही. आणि मला मधलं नाव **थेट** बघता यायला हवं (5वं नाव पटकन). माझी सध्याची रचना हे देते का? आणि रचना बदलली तर काय मिळतं- गमावतं?”

हाच पुढचा संशयित: **माहिती साठवायची रचना** (data structure). साधी यादी (array) थेट-बोट देते पण मध्ये घुसवणं महाग; दुसरी रचना (linked list) उलट. पहिली जोडी: पुढचा chapter.

Chapter 1.2 [SPINE]: संशायित: Array vs Linked List

चंदूचा प्रश्न: कुठली रचना थेट-बोट देते, कुठली मध्ये-घुसवणं सोपं? पहिल्या दोन data structures ची उलटतपासणी: array आणि linked list. दोन्ही “यादी” साठवतात, पण **विरुद्ध सौदे**.

Array: शेजारी-शेजारी खोल्या. array म्हणजे memory मध्ये सलग, शेजारी-शेजारी ठेवलेल्या खोल्या (नावं). हॉटेलच्या खोल्या 1, 2, 3... रांगेत. फायदा: **5व्या नावाला थेट बोट** (खोली-5 कुठे ते गणिताने कळतं: सुरुवात + 5): $O(1)$, क्षणात. Book 5 चा index/array हेच. तोटा: **मध्ये घुसवणं महाग**. खोली-3 आणि 4 च्या मध्ये नवं नाव? सगळ्या पुढच्या खोल्या एक-एक सरकवाव्या लागतात ($O(n)$): गर्दीच्या रांगेत मध्ये कुणाला घुसवण्यासारखं, सगळे मागे सरकतात.

Linked list: प्रत्येक खोली पुढच्याचा पत्ता सांगते. इथे खोल्या शेजारी नसतात; कुठेही विखुरलेल्या, पण प्रत्येक खोली **पुढच्या खोलीचा पत्ता** ठेवते (साखळी). खजिन्याच्या शोधासारखं: प्रत्येक चिठ्ठी पुढच्या चिठ्ठीचा पत्ता देते. फायदा: **मध्ये घुसवणं सोपं** ($O(1)$): फक्त दोन पत्ते बदला (आधीची चिठ्ठी नव्याकडे, नवी पुढच्याकडे), कुणी सरकत नाही. तोटा: **5व्या नावाला थेट बोट नाही** ($O(n)$): पहिल्या चिठ्ठीपासून पत्ते फॉलो करत जावं लागतं, उडी मारता येत नाही.

सौदा स्पष्ट (सूत्र):

	थेट-बोट (शोध)	मध्ये घुसवणं/काढणं
Array	$O(1)$ जलद	$O(n)$ महाग
Linked list	$O(n)$ महाग	$O(1)$ जलद

एकच “यादी”, दोन विरुद्ध सौदे. निवड कशावर? **तुम्ही जास्त काय करता:** थेट-बोट (वाचन, शोध) की मध्ये-बदल (घुसवणं-काढणं)? चंदूच्या फोन-वहीत: शोध खूप, मध्ये-बदल कमी -> **array बरं** (थेट-बोट जलद). पण जिथे सतत मध्ये-घुसवणं (उदा गाण्यांची सतत बदलणारी यादी) -> linked list. Book 5 चा वाचन-वि.-लेखन सौदा (1.3) इथे रचनेच्या पातळीवर, पुन्हा.

पण चंदूचा खरा प्रश्न वेगळा होता: शोध. array ने थेट-बोट दिलं, पण फक्त **क्रमांकाने** (5वं नाव); **नावाने** शोध (चंदूला हवं ते) अजून $O(n)$ च आहे (कुठलं नाव 5व्या खोलीत माहीत नाही, चाळावंच लागतं). म्हणजे array ने अर्धा प्रश्न सुटला (थेट- बोट), पण “रमेश कुठे” अजून हळू. चंदूला हवंय: **नाव दिलं की थेट खोली-क्रमांक**. ही जादुई रचना पुढच्या chapter मध्ये.

इथे लोक काय चुकीचं समजतात

”यादी म्हणजे यादी, रचना काय फरक पडतो.” प्रचंड फरक: तीच ”यादी” array मध्ये की linked list मध्ये यावर एकच कृती $O(1)$ की $O(n)$ होते: 1000 पट फरक मोठ्या data वर. आणि नवखे बहुधा **डिफॉल्ट** रचना (जी भाषा देते) वापरतात, ”मी जास्त काय करतो” न विचारता: आणि मग चुकीच्या सौद्याने app हळू. रचना निवडणं = ”शोध जास्त की बदल जास्त” हा एक प्रश्न विचारणं.

नकाशावर

- 1.1 Big-O = वाढीचा दर; चंदूचा app $O(n)$ शोध
- 1.2 array (थेट-बोट जलद, मध्ये-बदल महाग) vs linked list (उलट); ”शोध जास्त की बदल जास्त” ने निवडा

सूत्रावर: array-vs-linked-list हा शुद्ध सौदा: एकीत वेग इथे, दुसरीत तिथे: दोन्ही एकत्र फुकट मिळत नाही. रचना = कुठला वेग विकत, कुठला विकून हा निर्णय.

स्वतः बघा (5 मिनिटं)

दोन प्रकारे वस्तू ठेवा: (1) रांगेत, क्रमांकित खोल्यांत (array): ”तिसरा खोका” पटकन, पण मध्ये नवा खोका घालायला सगळे सरकवा. (2) प्रत्येक वस्तूला ”पुढची कुठे” ची चिठ्ठी (linked list): मध्ये घालणं सोपं, पण ”तिसरी वस्तू” शोधायला चिठ्ठ्या फॉलो करा. तुमच्या रोजच्या साठवणीत तुम्ही कुठली वापरता, आणि का?

विचार करा

चंदूला array ने थेट-बोट दिलं, पण **नावाने** शोध अजून हळू. तो विचारतो: ”मला हवंय: ’रमेश’ टाईप केलं की **थेट**, चाळल्याशिवाय, त्याची खोली. जणू प्रत्येक नावाला एक जादुई पत्ता आधीच माहीत असावा. हे शक्य आहे का: नाव -> थेट जागा, $O(1)$ मध्ये?”

शक्य आहे, आणि ती data structures मधली सर्वात जादुई, सर्वात वापरली जाणारी रचना आहे: **hash table**. नाव -> थेट जागा, चाळल्याशिवाय. चंदूच्या केसचा मोठा सुगावा: पुढचा chapter.

Chapter 1.3 [SPINE]: मुख्य सुगावा: Hash Table: जादूची वही

चंदूला हवय: नाव -> थेट जागा, चाळल्याशिवाय. आणि data structures मधली सर्वात जादुई रचना हेच देते: **hash table**. हा चंदूच्या केसचा सर्वात मोठा सुगावा आहे, आणि तुमच्या रोजच्या programming चं सर्वात वापरलं जाणारं हत्यार.

कल्पना: नावाचं गणिती पत्त्यात रूपांतर. समजा एक जादुई यंत्र आहे: तुम्ही “रमेश” टाकता, ते एक **आकडा** काढतं (उदा 4517): आणि तुम्ही रमेशची माहिती नेहमी खोली-4517 मध्येच ठेवता. पुढच्या वेळी “रमेश” शोधायचं? तेच यंत्र पुन्हा 4517 काढतं, थेट तिथे बघा: **चाळणं शून्य, $O(1)$!** त्या जादुई यंत्राचं नाव **hash function**: कुठलाही शब्द -> एक ठराविक आकडा (पत्ता). गावातल्या पोस्ट-वाटपाची उपमा: पत्त्यावरून थेट घर, गल्ली-गल्ली फिरण्याची गरज नाही. Hash table = नाव-वरून-थेट- जागा देणारी वही.

हे सगळीकडे आहे. तुम्ही programming मध्ये “dictionary”, “map”, “object” वापरता ते hash table च. Python चं dict, नाव->नंबर जोडी: सगळं $O(1)$ शोध. Book 4 चं memory, Book 5 चा cache: आत बहुधा hash table. “**नाव दिलं की थेट माहिती**” जिथे-जिथे लागतं तिथे hash table: आणि ते जवळजवळ सगळीकडे लागतं. म्हणून ही सर्वात वापरली जाणारी रचना.

पण सूत्र: जादू फुकट नाही: दोन किंमती. (1) **जागा:** hash table ला भरपूर रिकाम्या खोल्या लागतात (गर्दी झाली की मंदावतं): वेग विकत, **जागा** विकून (सूत्र पुन्हा). (2) **टक्कर (collision):** जादुई यंत्र कधीकधी **दोन वेगळ्या** नावांना **तोच** आकडा देतं (रमेश आणि सुरेश दोघांनाही 4517!): याला collision म्हणतात. मग त्या एका खोलीत दोघांना ठेवावं लागतं (छोटी linked list: 1.2!), आणि शोधताना त्या दोघांत चाळावं लागतं. जर hash function वाईट असेल (खूप collisions), तर hash table चा $O(1)$ हळूहळू $O(n)$ कडे घसरतो: जादू संपते. म्हणून **चांगलं hash function** (नावं समान वाटणारं, कमी collisions) हा या रचनेचा प्राण.

चंदूच्या केसचा मोठा तुकडा सुटला. चंदूने आपली फोन-वही hash table मध्ये टाकली: नाव -> थेट नंबर, $O(1)$. 10 लाख नावं, तरी शोध **milliseconds** मध्ये. 8 सेकंदांचा शोध क्षणार्धात. संशयित ($O(n)$ चाळणं) पकडला **आणि** सुधारला. पण गुप्तहेर सावध: “hash table ने **नेमकं-नाव** शोध सोडवला. पण चंदूला कधी ‘र’ ने सुरू होणारी **सगळी** नावं, किंवा **क्रमाने** नावं हवी असतील तर? hash table विस्कळीत (क्रम नाही) आहे: तिथे ती अडेल.” तपास पुढे चालू.

इथे लोक काय चुकीचं समजतात

"Hash table सगळ्यावर उत्तर, नेहमी $O(1)$." दोन इशारे: (1) hash table **क्रम ठेवत नाही** (नावं विस्कळीत): "क्रमाने द्या" किंवा "र-ते-श मधली सगळी" यांना ती निरुपयोगी (तिथे tree: 1.4). (2) वाईट hash function किंवा खूप गर्दी = collisions = $O(1)$ घसरून $O(n)$: "जादू" तुटते. Hash table अप्रतिम **नेमक्या-शोधाला**; पण "सगळं hash table मध्ये टाका" म्हणणारा क्रम-गरज आणि collision विसरतो. योग्य कामाला योग्य रचना.

नकाशावर

- 1.1 Big-O 1.2 array vs linked list
- 1.3 hash table = नाव -> थेट जागा ($O(1)$), जादूची वही); किंमत:
जागा + collisions; चंदूचा नेमका-शोध सुटला! पण क्रम नाही

सूत्रावर: hash table हा सूत्राचा सर्वात नाट्यमय सौदा: जवळजवळ- जादुई वेग ($O(1)$), पण भरपूर जागा आणि collision-जोखीम विकून. आणि तो क्रम विकतो: वेग घेतला, क्रम गमावला. फुकट काही नाही.

स्वतः बघा (5 मिनिटं)

तुमच्या फोनमधले contacts: नाव टाईप करताच तो **थेट** दिसतो (चाळत नाही): मागे hash-सदृश रचना. आता "स-ने सुरू होणारे सगळे" बघा: ती वेगळी (क्रमाने लावलेली) यादी लागते. दोन वेगळ्या गरजा, दोन वेगळ्या रचना: तुमचा फोन दोन्ही वापरतो, नकळत.

विचार करा

चंदूचा नेमका-शोध सुटला, पण नवी गरज: "मला 'र' ते 'श' मधली सगळी नावं, **क्रमाने**, पटकन हवीत (उदा एका भागातले सगळे लोक). Hash table विस्कळीत, array क्रमाने पण मध्ये-बदल महाग. अशी रचना आहे का जी **क्रम ठेवते आणि तरी जलद शोध-घुसवणं** दोन्ही देते?"

आहे, आणि ती निसर्गातल्या एका आकारावर बेतलेली आहे: **झाड** (tree). क्रम ठेवतं, आणि शोध-घुसवणं दोन्ही $O(\log n)$. पुढचा संशयित: tree.

Chapter 1.4 [SPINE]: संशायित: Tree: वंशावळ जी क्रम ठेवते

चंदूला क्रम आणि वेग दोन्ही हवेत. उत्तर एका सुंदर रचनेत: **tree** (झाड). आणि ते वंशावळीसारखं (कुळ) बांधलेलं असतं: वर एक, खाली फांद्या-फांद्या.

कल्पना: प्रत्येक निर्णयावर अर्ध-अर्ध. झाडाची रचना: वर एक नाव (मूळ), त्याखाली डावी-उजवी दोन फांदी. नियम: **डावीकडे लहान, उजवीकडे मोठं** (अकारविल्हे). “रमेश” शोधायचं? मुळापासून: मुळातलं नाव रमेशपेक्षा मोठं? डावीकडे जा; लहान? उजवीकडे. प्रत्येक पावलावर **अर्ध झाड कापलं जातं** (फोनबुकातलं अर्ध- अर्ध: 1.1). म्हणून 10 लाख नावं फक्त ~20 पावलांत: **$O(\log n)$** , शोध आणि घुसवणं दोन्ही. आणि hash-पेक्षा वेगळं: झाड **क्रम ठेवतं** (डावीकडून उजवीकडे वाचलं तर अकारविल्हे यादी): म्हणून “र ते श मधली सगळी” सहज मिळतात. चंदूची नवी गरज सुटली.

सौदा (सूत्र):

	नेमका-शोध	क्रमाने-यादी	घुसवणं
hash table	$O(1)$ जलद	नाही!	$O(1)$
tree (संतुलित)	$O(\log n)$	हो, सहज	$O(\log n)$

Hash नेमक्या-शोधाचा राजा (जलद पण क्रमहीन); tree क्रम-हवा- तिथला राजा (थोडं हळू पण क्रमबद्ध). पुन्हा: **गरजेनुसार निवड, एक सर्वोत्तम नाही.**

पण एक महत्वाचा इशारा: झाड “संतुलित” हवं. जर नावं आधीच क्रमाने आली (अ, ब, क, ड...) आणि तशीच झाडात घातली, तर झाड **एका बाजूला वाकडं** होतं: फांद्या न फुटता एक लांब सोट (म्हणजे परत linked list!): शोध पुन्हा $O(n)$. म्हणून खरी झाडं **स्वतःला संतुलित** ठेवतात (फिरवून-सारखं करून: “self-balancing”, उदा red-black tree, B-tree): फांद्या दोन्ही बाजूंनी सारख्या, $O(\log n)$ टिकतो. Book 5 चं B-tree (2.3: database चं हृदय) हेच: disk- साठी बांधलेलं संतुलित झाड. **झाडाची ताकद संतुलनात; संतुलन गेलं की ताकद गेली.**

Tree सगळीकडे आहे: file-folders (folder मध्ये folder: झाड), database index (B-tree: Book 5), निर्णय-झाडं (AI मध्ये), HTML पानाची रचना, कंपनीची उतरंड. **“एकाखाली अनेक, अनेकाखाली अनेक”** ही रचना जिथे-जिथे तिथे tree. Book 5 चा data-modeling (नाती) आठवा: one-to-many म्हणजे झाडच.

इथे लोक काय चुकीचं समजतात

”Tree म्हणजे नेहमी $O(\log n)$, hash पेक्षा हळू म्हणून टाळा.” दोन चुका: (1) असंतुलित झाड $O(n)$ होतं (वर): संतुलन गृहीत धरू नका. (2) आणि tree ला “हळू” म्हणून टाळणं चूक: जिथे **क्रम** हवा (यादी, श्रेणी-शोध, “पुढचं-मागचं”) तिथे hash निरुपयोगी, tree च हवं. $O(\log n)$ हे “हळू” नाही: 10 लाखांना 20 पावलं अप्रतिम. निवड “जलद की हळू” नाही, “क्रम हवा की नको” यावर.

नकाशावर

1.1-1.3 Big-O | array/linked | hash

1.4 tree = वंशावळ; प्रत्येक पावलावर अर्ध कापणं ($O(\log n)$);
क्रम ठेवतं (hash पेक्षा वेगळं); संतुलित हवं (नाहीतर $O(n)$)

सूत्रावर: tree हा सौदा “थोडा वेग विकून क्रम विकत घेणं”: hash पेक्षा किंचित हळू, पण क्रमबद्ध. आणि संतुलन राखायला थोडी अधिक मेहनत (जागा/कृती): फुकट क्रम नाही.

स्वतः बघा (5 मिनिटं)

तुमच्या फोनमधली files/folders बघा: folder मध्ये folder मध्ये file: एक झाड. एका file पर्यंत पोहोचायला तुम्ही मुळापासून फांद्या निवडत जाता (Downloads -> 2026 -> photos): प्रत्येक पावलावर बाकीच्या फांद्या “कापल्या.” हेच tree-शोध. तुम्ही रोज झाडावर चालता, नकळत.

विचार करा

चंदूचं app आता जलद: hash नेमक्या-शोधाला, tree क्रमाला. पण एक नवी गरज आली: “मला ‘रमेश कोणाकोणाला ओळखतो, आणि ते पुढे कोणाला’ असं **नात्यांचं जाळं** शोधायचंय (सामाजिक जोडण्या). हे झाड नाही: इथे कुणीही कुणाशीही जोडलेलं, वर-खाली नाही. अशी गुंतागुंतीची जोडणी कशी साठवायची-शोधायची?”

झाड म्हणजे “प्रत्येकाला एकच पालक”; पण खऱ्या जगात जोडण्या कुठूनही कुठेही: त्याला **graph** (आलेख/जाळं) म्हणतात. नकाशे, मैत्री, internet: सगळं graph. पुढचा संशयित.

Chapter 1.5 [SPINE]: संशायित: Graph: जोडण्यांचं जाळं

चंदूला नात्यांचं जाळं हवंय: कुणीही कुणाशीही. ही रचना म्हणजे **graph** (आलेख/जाळं), आणि ती जगातल्या सर्वात शक्तिशाली कल्पनांपैकी एक आहे: कारण अर्ध जग जाळं आहे.

Graph म्हणजे काय. टिंबं (गोष्टी: माणसं, शहरं, पानं) आणि त्यांना जोडणाऱ्या **रेषा** (नाती: मैत्री, रस्ते, links). Tree हा एक खास graph (प्रत्येकाला एकच पालक, वर-खाली); पण खरा graph मुक्त: रमेश-सुरेश जोडलेले, सुरेश-गणेश, गणेश-रमेश (चक्र!), कुणीही कुणाशी. गावांच्या नकाशाची उपमा: गावं (टिंबं) रस्त्यांनी (रेषा) जोडलेली, कुठूनही कुठे. **जिथे “गोष्टी आणि त्यांच्यातली नाती” तिथे graph.**

Graph सगळीकडे आहे (आणि तुम्ही रोज वापरता): (1) **नकाशे:** Google Maps = शहरं (टिंबं) + रस्ते (रेषा); “जवळचा रस्ता” म्हणजे graph मध्ये शोध. (2) **सामाजिक जाळं:** माणसं + मैत्री; “तुमच्या ओळखीचे-ओळखीचे” = graph. (3) **Internet:** पानं + links; Google चा शोध graph वर चालतो. (4) **AI:** Book 7 चं neural network एक graph आहे (neurons + जोडण्या)! **graph समजला की अर्ध आधुनिक tech उघडतं.**

दोन मुख्य शोध-पद्धती (graph मधून फिरणं): चंदूला “रमेश ते गणेश वाट आहे का” शोधायचं. दोन रीती:

रुंदीने (BFS): रमेशचे आधी सगळे थेट-मित्र बघा, मग त्यांचे मित्र, मग त्यांचे... लाटेसारखं बाहेर पसरत. “सर्वात कमी पावलांची वाट” शोधायला उत्तम (जवळचा मित्र-मार्ग).

खोलीने (DFS): एका मित्राच्या मागे खोल जात रहा (रमेश -> सुरेश -> त्याचा मित्र -> ...) टोक येईपर्यंत, मग मागे फिरून दुसरी वाट. “वाट आहे का” किंवा भूलभुलैया सोडवायला.

दोन्ही graph “पिंजून” काढतात, वेगवेगळ्या क्रमाने. आणि इथे सावधगिरी: graph मध्ये **चक्रं** असतात (रमेश->सुरेश->गणेश-> रमेश): म्हणून “कुठे जाऊन आलो” ची **खूण** ठेवावी लागते (visited), नाहीतर तेच-तेच फिरत अनंत loop. (Tree मध्ये चक्र नाही, म्हणून हे लागत नाही: graph ची अधिक गुंतागुंत.)

सूत्र इथे: graph = प्रचंड लवचिकता (कुठलीही जोडणी) विकत, **गुंतागुंत** विकून. Tree सोपं (एकच पालक, चक्र नाही); graph शक्तिशाली पण गुंतागुंतीचं (चक्रं, अनेक वाटा, महाग शोध). “जवळचा रस्ता” सारखे graph-प्रश्न अवघड-आणि-महाग होऊ शकतात (म्हणून Google Maps ला हुशार algorithms लागतात). शक्ती आणि गुंतागुंत एकत्र येतात.

इथे लोक काय चुकीचं समजतात

”Graph म्हणजे आलेख (bar chart, रेखा-आलेख).” पूर्ण वेगळं! इथे graph = टिंबं + जोडण्या (जाळं), आकडेवारीचा आलेख नाही. आणि दुसरा: “माझ्या app ला graph कशाला.” पण जिथे-जिथे “गोष्टी आणि नाती” (users आणि follows, products आणि “यासोबत घेतलं”, पानं आणि links) तिथे graph लपलेलं: ते ओळखता आलं की शिफारसी, शोध, जोडण्या: सगळं नव्या नजरेने दिसतं. Graph “विशेष” नाही; तो सर्वत्र आहे, फक्त नाव माहीत नसतं.

नकाशावर

1.1-1.4 Big-O | array/linked | hash | tree
 1.5 graph = टिंबं + जोडण्या (जाळं); नकाशे/मैत्री/internet/AI;
 BFS (रुंदी, जवळची वाट) / DFS (खोली); चक्रं -> visited-खूण

सूत्रावर: graph हा सौदा “अमर्याद जोडणी-शक्ती विकत, गुंतागुंत विकून.” सर्वात लवचिक रचना, म्हणून सर्वात गुंतागुंतीची: शक्ती आणि किंमत नेहमी सोबत.

स्वतः बघा (5 मिनिटं)

Google Maps उघडा आणि दोन ठिकाणांतला रस्ता बघा: Maps हजारो रस्त्यांच्या graph मधून “सर्वात कमी वेळाची वाट” शोधतं (BFS-कुळाचं). आणि “mutual friends” (सामाईक मित्र) कुठल्याही social app मध्ये = graph मधली सामायिक जोडणी. तुम्ही रोज graph-शोध वापरता, फक्त आत जाळं आहे हे दिसत नाही.

विचार करा

चंदूकडे आता सगळ्या रचना आहेत: array, hash, tree, graph. पण एक गोष्ट खटकते: “मला अनेकदा नावं **क्रमाने लावायची** असतात (यादी, अहवाल). हे ‘क्रमाने लावणं’ (sorting) नक्की कसं होतं, आणि ते हळू की जलद? 10 लाख नावं क्रमाने लावायला किती वेळ, आणि तिथे पुन्हा Big-O चा खेळ आहे का?”

आहे, आणि तो सुंदर आहे: क्रमाने लावण्याच्या (sorting) अनेक पद्धती, आणि त्यांच्यात $O(n^2)$ चा मृत्यू-सापळा आणि $O(n \log n)$ ची सुटका. Sorting ची शर्यत: पुढचा chapter.

Chapter 1.6 [SPINE]: Sorting ची शयत आणि Searching

चंदूचा प्रश्न: क्रमाने लावणं (sorting) कसं, आणि किती जलद? हा chapter दोन मूलभूत कामं जोडतो: लावणं (sort) आणि शोधणं (search), आणि त्यात Big-O चा जिवंत खेळ दाखवतो.

Sorting: क्रमाने लावणं: दोन जगं. 10 लाख नावं अकारविल्हे लावायची. दोन पद्धती, दोन Big-O:

भोळी पद्धत ($O(n^2)$): प्रत्येक नाव प्रत्येक दुसऱ्याशी तुलना, अदलाबदल करत. पत्त्यांचा गट्टा एकेक बघत जागेवर ठेवणं.
 10 नावं: 100 तुलना (ठीक). 10 लाख: 10 लाख x 10 लाख = खर्व तुलना: तासन्तास! मृत्यू-सापळा.
 हुशार पद्धत ($O(n \log n)$): "फोडा आणि जिंका": गट्टा अर्धा करा, प्रत्येक अर्धा वेगळा लावा (तोच नियम, लहान गट्ट्यावर), मग दोन लावलेले अर्धे एकत्र मिसळा (merge). 10 लाख: ~2 कोटी कृती (सेकंदाचा भाग). सुटका!

फरक प्रचंड: तीच नावं, $O(n^2)$ ने तास, $O(n \log n)$ ने सेकंद. आणि गंमत: जगातली सगळी तयार sort-साधनं (प्रत्येक भाषेतलं `sort`) $O(n \log n)$ च वापरतात: कारण $O(n^2)$ मोठ्या data ला अशक्य. म्हणून तयार `sort` वापरा, स्वतःचं भोळं लिहू नका: हा एक व्यावहारिक धडा हजारो तास वाचवतो.

"फोडा आणि जिंका" (divide and conquer): एक मोठं तत्त्व. हुशार sort मागचं तत्त्व सर्वत्र आहे: मोठी समस्या अर्धी-अर्धी फोडा, प्रत्येक तुकडा सोडवा, मग जोडा. हेच "अर्ध-अर्ध कापणं" (1.1 चा $O(\log n)$), तेच tree (1.4), तेच पुढचं recursion (1.8). एक कल्पना, अनेक जागी: आणि ती ओळखता येणं म्हणजे खोल नजर.

Searching: शोधणं: क्रमाची किंमत वसूल. एकदा नावं क्रमाने लावली, की शोध अर्ध-अर्ध कापत (binary search): "मधलं नाव बघा; हवं ते आधी? डावा अर्धा; नंतर? उजवा अर्धा": प्रत्येक पावलावर अर्धं गळतं: $O(\log n)$ (1.1 चं फोनबुक). 10 लाख नावं, ~20 पावलं. पण अट: **यादी क्रमाने हवी** (म्हणून आधी sort). म्हणजे: एकदाची महाग sort ($O(n \log n)$) केली की मग असंख्य शोध स्वस्त ($O(\log n)$ प्रत्येक): **एकदा गुंतवा, वारंवार कमवा.** Book 5 चा index हेच करतो (क्रमाने ठेवून जलद शोध).

चंदूला निवडीचा नियम मिळाला: एकदाच शोधायचं? sort ची मेहनत वाया (नुसतं चाळा, $O(n)$). वारंवार शोधायचं? आधी sort (किंवा tree/hash), मग प्रत्येक शोध स्वस्त. **शोध किती वेळा होणार** यावर रचना ठरते: पुन्हा "वाचन जास्त की एकदाच" (Book 5 चा आत्मा).

इथे लोक काय चुकीचं समजतात

”मी माझं sorting स्वतः लिहितो, सोपं आहे.” भोळं sorting लिहिणं सोपं, पण ते बहुधा $O(n^2)$ निघतं: छोट्या data ला चालतं, मोठ्यावर app गोठतं (आणि हा bug फक्त data वाढल्यावर दिसतो: सर्वात कपटी). जगातल्या हुशार लोकांनी दशकं घालवून $O(n \log n)$ sort बनवले आणि प्रत्येक भाषेत घातले: तयार **sort** वापरा. “स्वतः लिहिणं” ही हौस इथे महाग: चाकाचा पुनर्शोध, आणि वाकडं चाक.

नकाशावर

1.1-1.5 Big-O | array/linked | hash | tree | graph
 1.6 sorting: भोळं $O(n^2)$ मृत्यू-सापळा vs हुशार $O(n \log n)$ (फोडा-जिंका); तयार sort वापरा; searching: क्रमाने -> binary search $O(\log n)$ (एकदा sort, वारंवार स्वस्त शोध)

सूत्रावर: sort/search हा सौदा “एकदाची मेहनत (sort) विकत, पुढचे असंख्य शोध स्वस्त करून घेणं.” आणि $O(n^2)$ टाळणं म्हणजे “थोडी हुशारी (फोडा-जिंका) विकत, प्रचंड वेळ विकत घेणं.” महागडं ओळखणं इथे थेट तास वाचवतं.

स्वतः बघा (5 मिनिटं)

पत्त्यांचा गट्टा (किंवा कागद) क्रमाने लावा आणि स्वतःकडे बघा: तुम्ही बहुधा “फोडा-जिंका” किंवा “जागेवर घालत जाणं” वापरता. मग एका लावलेल्या गट्ट्यात एक पत्ता शोधा: तुम्ही मधे बघून अर्ध-अर्ध कापता (binary search), पहिल्यापासून चाळत नाही. तुमचा मेंदू आधीच हुशार algorithms वापरतो; त्यांना नाव देणं इतकंच उरलं.

विचार करा

चंदूला “फोडा आणि जिंका” आवडलं: मोठी समस्या अर्धी करून, तोच नियम लहान तुकड्यावर लावणं. पण त्याला गोंधळ: “तोच नियम **स्वतःच्याच** लहान आवृत्तीला लावणं: हे विचित्र वाटतं. एक function स्वतःलाच बोलावतं? हे कसं चालतं, आणि अनंत loop होत नाही का?”

हे “स्वतःला बोलावणं” म्हणजे recursion: programming मधली सर्वात सुंदर आणि सर्वात गोंधळवणारी कल्पना. आरशातला आरसा. ते कसं चालतं आणि अनंत का होत नाही: पुढचा chapter.

Chapter 1.7 [SPINE]: Recursion आणि आठवणीने पुनर्वापर (DP)

चंदूचा गोंधळ: function स्वतःलाच बोलावतं, अनंत होत नाही का? हा chapter दोन जोडलेल्या सुंदर कल्पना देतो: recursion (स्वतःला बोलावण) आणि dynamic programming (आठवणीने पुनर्वापर).

Recursion: आरशातला आरसा. recursion म्हणजे एक समस्या स्वतःच्याच लहान आवृत्तीत सोडवणं. रशियन बाहुल्यांची (एकात एक) उपमा: मोठी बाहुली उघडा, आत लहान, आत अजून लहान... शेवटी सर्वात लहान (जिच्यात काही नाही: थांबा). recursion चे दोन भाग, आणि दुसरा चुकला की अनंत loop:

1. लहान करत जाणं: "10 चा factorial = 10 x (9 चा factorial)";
"9 चा = 9 x (8 चा)"... प्रत्येक वेळी समस्या लहान.
2. थांबा (base case): "1 चा factorial = 1" (इथे थांबा, पुढे बोलावू नको).

थांबा असेल तर recursion सुंदर संपतं (सर्वात लहान बाहुलीत थांबतं); थांबा विसरला तर अनंत loop (बाहुल्या कधीच संपत नाहीत, यंत्र कोसळतं). चंदूच्या प्रश्नाचं उत्तर: **base case मुळे अनंत होत नाही.** आणि recursion "फोडा-जिंका" (1.6), tree-फिरणं (1.4), graph-शोध (1.5): सगळीकडे: कारण त्या सगळ्यांचा आकार "स्वतःसारखा लहान" असतो.

पण recursion चा एक कपटी सापळा: तेच-तेच पुन्हा मोजणं. प्रसिद्ध उदाहरण: Fibonacci (प्रत्येक संख्या आधीच्या दोनची बेरीज: 1,1,2,3,5,8...). recursion ने "10वी = 9वी + 8वी"; पण "9वी" काढायला परत "8वी + 7वी"... आणि "8वी" दोनदा मोजली जाते, "7वी" तीनदा... तेच-तेच काम पुन्हा-पुन्हा: झाड स्फोटक फुगतं, $O(2^n)$: 40वी संख्या काढायला **अब्जावधी** कृती, मिनिटं! भोळं recursion इथे मृत्यू-सापळा.

उपाय: आठवणीने पुनर्वापर (dynamic programming). कल्पना साधी: **एकदा मोजलेलं लिहून ठेवा, पुन्हा लागलं तर आठवणीतून घ्या, नव्याने मोजू नका.** "8वी संख्या" एकदा मोजली, वहीत लिहिली; पुढे लागली तर वहीतून (मोजणं नको). आता प्रत्येक संख्या **एकदाच** मोजली जाते: $O(2^n)$ चा $O(n)$ होतो: अब्जावधी कृती काही मोजक्या! व्यापाऱ्याची उपमा: रोजचा भाव पुन्हा-पुन्हा गोदामात जाऊन न बघता एकदा वहीत (Book 5 चा cache, 1.4! इथे algorithm मध्ये). **DP = recursion + आठवण (memoization): तेच काम दोनदा करू नको.**

सूत्र इथे स्पष्ट: DP = जागा (आठवणीची वही) विकत, वेळ (पुनरावृत्ती टाळून) विकत घेणं. classic time-space सौदा: थोडी memory खर्चून प्रचंड वेळ वाचवणं. आणि हेच पूर्ण Book 6 चं सूत्र (front matter): हुशारी = वेळ-जागा अदलाबदल. DP हे त्याचं सर्वात शुद्ध उदाहरण.

इथे लोक काय चुकीचं समजतात

”Recursion elegant आहे म्हणून नेहमी वापरा.” भोळं recursion (DP शिवाय) तेच-तेच मोजून स्फोटक हळू होऊ शकतं (Fibonacci $O(2^n)$): elegant दिसतं, पण मृत्यू-सापळा. आणि उलट: “recursion गोंधळाचं, टाळा.” काही समस्या (tree, फोडा-जिंका) recursion ने सर्वात स्वच्छ. नियम: recursion वापरा जिथे समस्या स्वतःसारखी लहान होते; पण तेच-तेच मोजलं जातंय का तपासा, असेल तर आठवण (DP) जोडा. Elegance आणि efficiency दोन वेगळ्या गोष्टी; दोन्ही बघा.

नकाशावर

1.1-1.6 Big-O | array/linked | hash | tree | graph | sort/search
 1.7 recursion = स्वतःला बोलावण (base case ने थांबा); सापळा:
 तेच-तेच मोजणं ($O(2^n)$); उपाय DP = आठवण (जागा विकत,
 वेळ घेत): हुशारी = वेळ-जागा अदलाबदल

सूत्रावर: DP हे सूत्राचं शुद्ध रूप: memory (जागा) खर्चून वेळ विकत घेणं. आणि recursion चा सापळा दाखवतो की “सुंदर” आणि “कार्यक्षम” वेगळे: नजर दोन्ही बघते.

स्वतः बघा (5 मिनिटं)

एक हिशेब दोनदा करायची वेळ आली की तुम्ही तो लिहून ठेवता (पुन्हा करत नाही): ते DP. आणि “आरशासमोर आरसा” (अनंत प्रतिबिंब) बघितलंय? ते recursion-शिवाय-थांबा: भीतीदायक अनंत. recursion ला base case म्हणजे “एका आरशावर पडदा”: प्रतिबिंब तिथे थांबतं. दोन्ही कल्पना रोजच्या अनुभवात आहेत.

विचार करा

चंदूला सगळ्या रचना आणि युक्त्या कळल्या. आता तो मागे वळून बघतो: “मी एकेक संशयित तपासला: $O(n)$ शोध, चुकीची रचना. पण खरं app मध्ये अनेक गोष्टी एकत्र चुकतात. मला एकदा, पूर्ण, बघायचंय: ‘ $O(n^2)$ ने खरोखर मेलेली systems’ कुठली, आणि त्यातून काय शिकायचं: म्हणजे माझ्या app मध्ये तो सापळा कधीच येणार नाही.”

हाच पुढचा chapter, विभागाचा शव-विच्छेदन: खऱ्या जगात $O(n^2)$ आणि चुकीच्या रचनांनी मेलेल्या systems, आणि त्यातून चंदूसाठी (आणि तुमच्यासाठी) कायमचे धडे. पुढे.

Chapter 1.8 [SPINE]: शव-विच्छेदन: $O(n^2)$ ने मेलेल्या Systems

विभागाचा शव-विच्छेदन: खऱ्या जगात चुकीच्या Big-O ने आणि चुकीच्या रचनांनी systems कशा मरतात, आणि चंदूसाठी कायमचे धडे. उलटा chapter: प्रेतापासून सुरुवात.

प्रेत 1: गर्दीत गुडघे टेकणारं app. सर्वात सामान्य मृत्यू: app launch ला जलद, यश आल्यावर (users वाढल्यावर) **कोसळतं**. कारण जवळजवळ नेहमी तेच: कुठेतरी एक $O(n^2)$ किंवा $O(n)$ लपलेलं जे लहान data ला दिसलं नाही (चंदूचा 8-सेकंद-शोध, मोठ्या प्रमाणात). उदा: “प्रत्येक user साठी सगळ्या users मधून काहीतरी शोधणं” ($O(n^2)$): 100 users ला ठीक, 1 लाख users ला यंत्र ठप्प. Flash-sale, viral moment: नेमक्या यशाच्या क्षणी app मरतं. **धडा: लहान-data-test फसवं; Big-O आधी बघा (1.1).**

प्रेत 2: database वर लपलेलं $O(n)$. Book 5 चा full scan (1.3) इथे परत: index विसरला, म्हणून प्रत्येक query पूर्ण table चाळते. एक query $O(n)$, आणि हजार queries एकाच वेळी: system गुडघ्यावर. अनेक startups चं पहिलं मोठं outage हेच: “index टाकायला विसरलो.” **धडा: जिथे शोध, तिथे योग्य रचना (index/ hash/tree), आधीच.**

प्रेत 3: N+1 सापळा (सूक्ष्म पण घातक). एक अतिशय सामान्य, लपलेला मृत्यू: “100 users ची यादी आणा” (1 query), मग प्रत्येक user साठी वेगळी query (“याचे orders आणा”): $1 + 100 = 101$ queries जिथे 2 पुरल्या असल्या. 100 ला सुसह्य, 10,000 users ला 10,001 queries: app रांगतं. हे code वाचून सहज दिसत नाही (प्रत्येक query निरुपद्रवी दिसते), पण एकत्र $O(n)$ स्फोट. **धडा: “हे loop मध्ये किती वेळा चालतंय” नेहमी विचारा; loop मधली एक query म्हणजे लपलेला $O(n)$.**

तिन्ही प्रेतांचा समान जंतू: प्रत्येकात एक कृती लहान-प्रमाणात स्वस्त दिसली, पण मोठ्या-प्रमाणात महाग निघाली: आणि ती फक्त यशाच्या क्षणी (गर्दी/data वाढल्यावर) फुटली: सर्वात महाग वेळी. म्हणून पूर्ण विभागाचा सर्वात मोठा धडा: “**आत्ता चालतंय” पुरेसं नाही; “10x, 100x वाढलं तर काय” हा प्रश्न आधी विचारा.** आणि तो विचारायला Big-O चा चष्मा लागतो: तोच या विभागाने दिला. Top engineer हा प्रश्न कोड लिहितानाच विचारतो; नवखा production मध्ये app मेल्यावर शिकतो (महाग शाळा).

चंदूची केस, इथवर: चंदूने $O(n)$ शोध पकडला (1.1), hash ने सोडवला (1.3), योग्य रचना निवडायला शिकला (1.2-1.5), $O(n^2)$ टाळायला शिकला (1.6-1.7). त्याचं app आता जलद. पण गुप्तहेर सावध: “हे सगळे संशयित **algorithm** च्या जगातले होते. पण app अजूनही कधीकधी हळू आहे: आणि आता algorithm बरोबर आहे. म्हणजे संशयित **खाली**, यंत्राच्या तळात आहे: memory, disk, CPU. तपास तिथे न्यावा लागेल.” Part 2 चं दार.

इथे लोक काय चुकीचं समजतात

”माझं app आता चालतंय, Big-O ची काळजी नंतर.” तिन्ही प्रेतं हेच सांगतात: “नंतर” म्हणजे “यशाच्या क्षणी कोसळणं.” $O(n^2)/N+1$ हे bugs testing मध्ये **कधीच** दिसत नाहीत (लहान data), फक्त production-गर्दीत: म्हणजे सर्वात वाईट वेळी, सर्वात जास्त users समोर. उलट टोकही चूक: “सगळं आधीच optimize करा” (premature: Book 5). नियम: **लहान ठेवा, पण Big-O बघा; जिथे आकार वाढणार तिथे आधीच योग्य रचना.** “चालतंय” आणि “वाढल्यावर चालेल” हे वेगळे प्रश्न.

नकाशावर

1.1-1.7 Big-O ... recursion/DP
 1.8 शव-विच्छेदन: गर्दीत-कोसळणं (लपलेला $O(n^2)$) | database
 full-scan (index विसरला) | $N+1$ सापळा (loop मधली query);
 “10x वाढलं तर?” आधी विचारा; संशयित आता तळात (Part 2)

सूत्रावर: पूर्ण विभाग एका वाक्यात: **महागडं कुठलं हे आकार- वाढण्याआधी ओळखणं = वेळ- जागेचा सौदा जाणून करणं.** आणि तो न ओळखणारे यशाच्या क्षणीच कोसळतात: सूत्र न पाळण्याची किंमत नेहमी सर्वात वाईट वेळी वसूल होते.

स्वतः बघा (5 मिनिटं)

कधी एखादं app/website गर्दीच्या वेळी (सण, निकाल, tatkal- तिकीट) हँग झालेलं आठवा: सामान्य वेळी ठीक, गर्दीत ठप्प. आता तुम्हाला कारण माहीत: कुठेतरी लपलेला $O(n)$ किंवा $O(n^2)$ जो गर्दीत स्फोटला. “एवढी मोठी company, गर्दीत का हँग” या प्रश्नाचं उत्तर आता Big-O च्या भाषेत देता येईल.

विचार करा

चंद्रूचं algorithm बरोबर, तरी app कधीकधी हळू. “म्हणजे अडचण code च्या **तर्कात** नाही, यंत्राच्या **प्रत्यक्ष कामात** आहे: memory किती वेगवान, disk किती हळू, CPU एका वेळी किती करतो. मला यंत्राचा **तळ** बघायला हवा: माझं code प्रत्यक्ष चालतं तेव्हा हार्डवेअरमध्ये नक्की काय घडतं?”

हाच Part 2 चा विषय: यंत्राचा तळ. memory ची उतरंड (का काही गोष्टी 1000 पट हळू), CPU प्रत्यक्ष कसं चालतं, आणि तुमचं code (Python) चालतं म्हणजे नक्की काय होतं. तपास हार्डवेअरच्या खोलीत जातो. Part 2.

BOOK 6, PART 1 चा शेवट

जे तुमच्याकडे आता आहे

Big-O	वाढीचा दर (आकार दुप्पट -> वेळ किती-पट); $O(1)/O(\log n)/O(n)/O(n^2)$; लहान-data फसवं
array/linked	array (थेट-बोट जलद, मध्ये-बदल महाग) vs linked (उलट); "शोध जास्त की बदल जास्त" ने निवडा
hash table	नाव -> थेट जागा $O(1)$ (जादूची वही); किंमत जागा + collision; क्रम ठेवत नाही
tree	वंशावळ; अर्थ-अर्थ ($O(\log n)$); क्रम ठेवत; संतुलित हवं (नाहीतर $O(n)$)
graph	टिंब + जोडण्या (जाळ); नकाशे/मैत्री/internet/AI; BFS/DFS; चक्रं -> visited-खूण; लवचिक पण गुंतागुंत
sorting/search	भोळं $O(n^2)$ मृत्यू-सापळा vs हुशार $O(n \log n)$; तयार sort वापरा; binary search $O(\log n)$
recursion/DP	स्वतःला बोलावण (base case); सापळा तेच-तेच मोजणं; DP = आठवण (जागा विकत, वेळ घेत)
शव-विच्छेदन	गर्दीत-कोसळणं/full-scan/ $N+1$; "10x वाढलं तर?" आधी विचारा

स्वतःची परीक्षा (4 प्रश्न, बोलून; उत्तरं खाली)

1. App test मध्ये जलद, म्हणजे कार्यक्षम का?

नाही: लहान data वर सगळे algorithms जलद ($O(n^2)$ सुद्धा). Big-O बघा: "10x, 100x वाढलं तर?" तेच खरं. (1.1, 1.8)

2. "नाव दिलं की थेट माहिती, $O(1)$ " कुठली रचना, आणि तिची मर्यादा?

hash table; पण क्रम ठेवत नाही (श्रेणी-शोधाला निरुपयोगी), आणि collision/गर्दीने $O(n)$ कडे घसरू शकते. (1.3)

3. क्रम आणि जलद शोध दोन्ही हवं: कुठली रचना, अट काय?

tree ($O(\log n)$, क्रमबद्ध); अट: संतुलित हवं, नाहीतर एका बाजूला वाकडं होऊन $O(n)$. (1.4)

4. DP (dynamic programming) चा गाभा, आणि कुठला सौदा?

तेच-तेच मोजू नको: एकदा मोजलेलं आठवणीत ठेव, पुन्हा घे. सौदा: जागा (memory) विकत, वेळ (पुनरावृत्ती टाळून) घेत. (1.7)

पुढच्या भागाची चाहूल

चंद्रचं algorithm बरोबर, तरी app कधी हळू: संशयित यंत्राच्या तळात. Part 2: यंत्राचा तळ: memory ची उतरंड (का काही 1000 पट हळू), CPU प्रत्यक्ष कसं चालतं, OS एका वेळी हजार कामं कशी करतं, आणि तुमचं Python code चालतं म्हणजे नक्की काय. तपास हार्डवेअरच्या खोलीत.

BOOK 6, PART 1 चा MINI-GLOSSARY

array	शेजारी-शेजारी खोल्या; थेट-बोट $O(1)$ (1.2)
Big-O	वाढीचा दर; आकार-वि.-काम (1.1)
binary search	क्रमबद्ध यादीत अर्ध-अर्ध शोध $O(\log n)$ (1.6)
BFS/DFS	graph रुंदीने / खोलीने पिंजणं (1.5)
collision	दोन नावांना तोच hash-आकडा (1.3)
divide & conquer	मोठं अर्ध-अर्ध फोडून सोडवणं (1.6)
DP	recursion + आठवण; तेच-तेच टाळणं (1.7)
graph	टिंबं + जोडण्या (जाळ) (1.5)
hash table	नाव -> थेट जागा $O(1)$ (1.3)
linked list	पत्ता-साखळी; मध्ये-घुसवणं $O(1)$ (1.2)
$O(n^2)$	चौरस; मृत्यू-सापळा मोठ्या data वर (1.1)
recursion	स्वतःला बोलावणं; base case ने थांबा (1.7)
tree	वंशावळ; $O(\log n)$, क्रमबद्ध, संतुलित हवं (1.4)